

Desarrollo de Gestión y Consulta de Tickets

Actividad N°: 21 **Fecha:** 29/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Implementar y documentar los endpoints de consulta de boletos, diferenciando el acceso general (Jefe) del acceso restringido por punto de trabajo (Staff).

2. Endpoints de consulta implementados

Endpoint	Acceso	Descripción
GET /api/tickets	jefe, staff	Consulta general con filtros; si es staff, se fuerza el filtro por su puntoTrabajo
GET /api/puntos-venta/:id/tickets	Cualquier autenticado	Tickets asociados a un punto de venta específico (para Jefe)
GET /api/puntos-venta/staff/tickets	staff, impresor	Tickets del punto de trabajo del usuario autenticado
GET /api/puntos-venta/:id/tickets/check-changes	Cualquier autenticado	Verificación ligera de cambios recientes (sincronización)
GET /api/puntos-venta/staff/tickets/check-changes	staff, impresor	Igual al anterior, restringido al punto de trabajo del staff
GET /api/puntos-venta/:id/estadisticas	Cualquier autenticado	Conteo de tickets por localidad dentro de un punto de venta

3. Regla de negocio central: aislamiento por punto de trabajo

Para un usuario Staff, el sistema **nunca** confía en un valor de punto de trabajo enviado desde el cliente: siempre se resuelve del lado del servidor a partir de **req.user.puntoTrabajo**, buscando el **PuntoVenta** correspondiente por nombre y usando sus localidades asociadas para construir el filtro de consulta. Esto asegura que un Staff no pueda ver boletos de otro punto de trabajo aunque manipule la petición HTTP.

4. Corrección aplicada: eliminación de fallback hardcodeado

Durante el desarrollo se identificó que **getTicketsForStaff** contenía un **mapeo de respaldo hardcodeado** (**puntoTrabajoLocalidades**) con nombres de localidades de un evento anterior del proyecto (ej. "Hips Don't Lie PLATINUM", "SOLTERA FAN ZONE"), que se activaba silenciosamente si el nombre del punto de trabajo del usuario no coincidía con ningún **PuntoVenta** real y vigente. Esto contradecía el principio de diseño de **localidades dinámicas** (Semana 2, RF-10) y podía mostrarle a un Staff mal configurado datos de un concierto distinto al actual, sin ninguna advertencia.

Archivo modificado: backend/src/controllers/puntoVentaController.js

```
- let localidadesAsignadas = [];  
-  
- if (puntoVenta) {  
-   localidadesAsignadas = puntoVenta.localidades;  
- } else {  
-   const puntoTrabajoLocalidades = {  
-     'boletería norte': ['GENERAL', 'PREFERENCIA'],  
-     'boletería sur': ['TRIBUNA', 'PALCO'],  
-     'centro comercial': ['Las Mujeres Facturan BOX', 'Antología GOLDEN'],  
-     'punto central': ['Hips Don\'t Lie PLATINUM', 'SOLTERA FAN ZONE'],  
-     'entrada principal': ['GENERAL', 'PREFERENCIA', 'TRIBUNA']  
-   };  
-   localidadesAsignadas = puntoTrabajoLocalidades[userPuntoTrabajo] ||  
+ ['GENERAL'];  
- }  
+ if (!puntoVenta) {  
+   return res.status(404).json({  
+     success: false,  
+     message: `No se encontró un punto de venta activo llamado  
"${userPuntoTrabajo}". Verifique la configuración del punto de trabajo del  
usuario.`  
+   });  
+ }  
+  
+ const localidadesAsignadas = puntoVenta.localidades;
```

Con este cambio, si el punto de trabajo de un usuario Staff está mal configurado (no coincide con ningún Punto de Venta activo), el sistema responde con un error explícito en vez de mostrar datos de un evento incorrecto.

5. Consideraciones de rendimiento aplicadas

- Uso de `.lean()` en consultas de solo lectura para reducir el overhead de hidratar documentos Mongoose completos.
- `maxTimeMS()` en todas las consultas de listado y conteo, para evitar que una consulta lenta bloquee el servidor indefinidamente.
- Paginación con límite máximo forzado de 100 elementos por página, sin importar lo que solicite el cliente.
- Ejecución en paralelo (`Promise.all`) de la consulta de datos y el conteo total, en vez de secuencial.

6. Conclusiones del día

Se implementaron los endpoints de consulta diferenciando correctamente el alcance de datos según el rol, y se corrigió una inconsistencia real de diseño (fallback hardcodeado de un evento anterior) que violaba el principio de localidades dinámicas acordado desde la Semana 2.

Observaciones: Corrección aplicada y verificada; sin observaciones adicionales.

Implementación de Búsquedas y Filtros

Actividad N°: 22 **Fecha:** 30/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

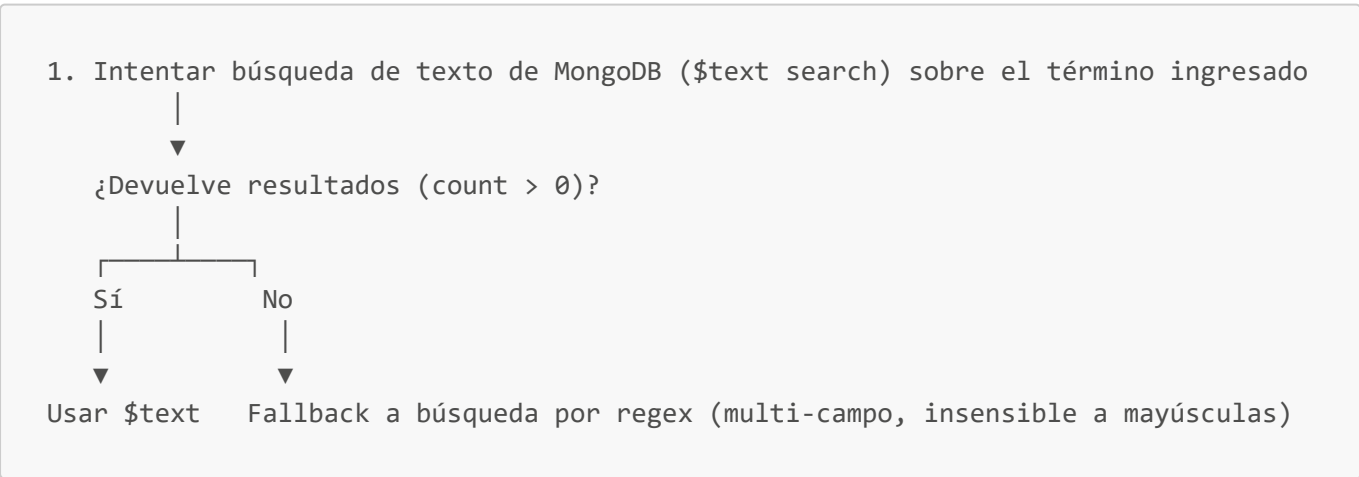
Implementar el motor de búsqueda y los filtros de la pantalla de tickets, cubriendo el criterio de aceptación de HU-02 (búsqueda por nombre, cédula, email o Ticket ID).

2. Parámetros de búsqueda soportados

Parámetro	Campo(s) afectado(s)	Tipo de coincidencia
search	First Name, Last Name, Email, Ticket ID, Transaction ID, Numero de Cedula	Texto general, insensible a mayúsculas (regex i)
ticketIdSearch	Ticket ID	Coincidencia exacta (string)
seatSearch	Seat	Texto parcial, insensible a mayúsculas
puntoTrabajo	puntoTrabajo	Exacto (solo aplicable/forzado para Jefe consultando; en Staff se ignora y se usa el propio)
sortBy / sortOrder	Cualquier campo ordenable	Ascendente/descendente

3. Estrategia de búsqueda en `getTicketsByPuntoVenta`

Se implementó una estrategia de dos niveles para optimizar el rendimiento en colecciones grandes:



Esto permite aprovechar índices de texto cuando existen y siguen siendo efectivos, sin dejar de funcionar si la búsqueda de texto no encuentra coincidencias (por ejemplo, búsquedas parciales de cédula que `$text` no maneja bien por defecto).

4. Combinación de filtros

Los filtros se combinan con operadores `$and/$or` de MongoDB según la cantidad de criterios activos: un único filtro se aplica directo; dos o más se combinan siempre con `$and` para que actúen como intersección (por ejemplo, "búsqueda general" y "localidad específica" al mismo tiempo), evitando el error común de que agregar un segundo filtro amplíe accidentalmente los resultados en vez de acotarlos.

5. Corrección aplicada: campo de cédula duplicado/muerto

Se identificó que las condiciones de búsqueda por cédula incluían dos nombres de campo distintos:

```
{ 'Numero de Cedula:': searchRegex },      // Coincide con el schema real
(Ticket.js)
{ 'Número de Cédula: ': searchRegex }      // Con tilde y espacio final – nunca
coincide con ningún documento
```

El segundo nunca puede producir coincidencias porque no existe ningún campo con ese nombre exacto en el esquema (`Ticket.js` define `'Numero de Cedula:'`, sin tilde). Era código muerto inofensivo (no generaba errores ni resultados incorrectos), pero agregaba ruido y confusión para quien mantenga el código a futuro.

Archivos corregidos: `backend/src/controllers/ticketController.js` y `backend/src/controllers/puntoVentaController.js` (3 ocurrencias en total: `getTickets`, `getTicketsByPuntoVenta`, `getTicketsForStaff`).

```
- { 'Numero de Cedula:': searchRegex },
- { 'Número de Cédula: ': searchRegex }
+ { 'Numero de Cedula:': searchRegex }
```

6. Frontend — filtros en `TicketsPage.jsx`

- Campo de búsqueda general con debounce para no disparar una petición por cada tecla presionada.
- Campo específico de búsqueda por localidad (`seatSearch`).
- Selector de punto de venta (solo visible para Jefe; el Staff no lo ve porque su punto de trabajo ya está fijo).
- Indicador de "usuario interactuando" para pausar actualizaciones en tiempo real mientras se escribe una búsqueda (ver Semana 1, RF-13).

7. Conclusiones del día

El motor de búsqueda queda implementado con una estrategia de dos niveles (texto + regex de respaldo) y se corrigió una inconsistencia de nombres de campo que, aunque no afectaba el resultado funcional, representaba deuda técnica innecesaria.

Observaciones: Corrección de campo duplicado aplicada y verificada; sin observaciones adicionales.

Desarrollo del Proceso de Canje de Entradas

Actividad N°: 23 **Fecha:** 01/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Implementar y documentar el proceso completo de canje de boletos (individual y masivo), integrando validación de datos, actualización de estado, auditoría y notificación en tiempo real.

2. Secuencia completa del canje individual (**canjeTicket**)

```

POST /api/tickets/:id/canje { quienRetira, parentesco?, quienOtro?, celular }
    |
    ▼
Validar: quienRetira y celular obligatorios
    |
    ▼
Si quienRetira = "Otro": validar parentesco y quienOtro obligatorios
    |
    ▼
Validar que quienRetira ∈ {Titular, Titular Compra, Otro}
    |
    ▼
Buscar ticket por 'Ticket ID' → 404 si no existe
    |
    ▼
¿ticket.canjeado === true? —Sí→ 400 "Este ticket ya fue canjeado"
    | No
    ▼
Actualizar: canjeado=true, fechaCanje=now, usuarioCanje, puntoCanje,
            quienRetira, celular (+ parentesco/quienOtro si aplica)
    |
    ▼
ticket.save()
    |
    ▼
Crear AuditLog { tipo: 'canje', ticketId, transactionId, detalles, ip }
            (si falla el log, NO se revierte el canje – se registra el error aparte)
    |
    ▼
Emitir Socket.IO 'ticket-updated' a sala punto-venta-{id} y staff-{puntoTrabajo}
    |
    ▼
200 { ticket actualizado }
  
```

3. Secuencia del canje masivo (**bulkCanjeTickets**)

```

POST /api/tickets/bulk-canje { ticketIds[], canjeData }
    |
    ▼
Validar ticketIds no vacío + campos obligatorios de canjeData
    |
    ▼
Buscar TODOS los tickets solicitados (canjeados y no canjeados)
    |
    ▼
Separar en dos grupos: ticketsToRedeem (pendientes) / alreadyRedeemed (ya
canjeados)
    |
    ▼
¿ticketsToRedeem vacío? —Sí→ 400 "Todos ya fueron canjeados"
    | No
    ▼
Construir un único updateData común (mismos datos de retiro para todos)
    |
    ▼
bulkWrite(): un updateOne por cada ticket pendiente, en una sola operación a
MongoDB
    |
    ▼
Insertar un AuditLog por cada ticket recién canjeado (insertMany, con
bulkOperation: true)
    |
    ▼
Emitir un evento Socket.IO 'ticket-updated' por cada ticket actualizado
    |
    ▼
200 { processed, updated, alreadyRedeemed, total, tickets }

```

4. Decisiones de implementación relevantes

- **Idempotencia parcial en canje masivo:** si dentro de la selección hay tickets ya canjeados, la operación no falla por completo; simplemente los excluye y reporta cuántos se procesaron y cuántos ya estaban canjeados. Esto evita que un Jefe deba "limpiar" manualmente su selección antes de reintentar.
- **Auditoría no bloqueante:** un fallo al escribir el log de auditoría se captura en un bloque `try/catch` separado y **no revierte** el canje ya guardado; se prioriza que la operación de negocio (entregar la entrada) no se pierda por un problema secundario de logging.
- **bulkWrite en vez de N operaciones save() secuenciales:** para lotes grandes (compras corporativas), reduce drásticamente la cantidad de round-trips a la base de datos.
- **Notificación en tiempo real por ticket individual:** incluso en el canje masivo, cada ticket emite su propio evento `ticket-updated`, para que otros usuarios vean los boletos pintarse de verde uno por uno en la interfaz, en vez de una actualización masiva abrupta.

5. Alineación con historias de usuario

Historia de usuario	Cobertura
HU-03 — Canje individual de boleto	✓ Implementada (<code>canjeTicket</code>)
HU-04 — Canje masivo de boletos	✓ Implementada (<code>bulkCanjeTickets</code> , restringida a Jefe desde el fix de la Semana 4)
HU-09 — Actualización en tiempo real	✓ Implementada (emisión Socket.IO en ambos flujos)

6. Conclusiones del día

El proceso de canje queda completamente implementado, tanto en su variante individual como masiva, cumpliendo las reglas de negocio definidas desde la Semana 1 (bloqueo de doble canje, datos obligatorios de quien retira) y conservando trazabilidad y tiempo real en ambos casos.

Observaciones: Sin observaciones.

Implementación de Impresión y Validación de Tickets

Actividad N°: 24 **Fecha:** 02/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Implementar el flujo de impresión de boletos y su reimpresión controlada, con las validaciones de datos correspondientes (HU-05 y RF-07).

2. Impresión (**printTicket**)

```
POST /api/tickets/:id/print { quienRetira, quienOtro?, parentesco?, celular }
|
▼
Validar: quienRetira y celular obligatorios
|
▼
Si quienRetira = "Otro": validar quienOtro y parentesco obligatorios
|
▼
Buscar ticket por 'Ticket ID' → 404 si no existe
|
▼
{ticket.impreso === true Y usuario NO es Jefe? —Sí→ 400 "Este ticket ya fue impreso"
| No (o es Jefe reimprimiendo)
▼
Actualizar: impreso=true, fechaImpresion=now, usuarioResponsable, puntoTrabajo,
quienRetira, celular (+ quienOtro/parentesco si aplica)
|
▼
ticket.save()
|
▼
Emitir Socket.IO 'ticket-updated' (action: 'print')
|
▼
200 { ticket actualizado }
```

Nota de diseño: a diferencia del canje (**canjeTicket**, que bloquea totalmente un segundo intento), la impresión permite que un **Jefe** reimprima directamente desde el mismo endpoint si es necesario, mientras que un Staff solo puede imprimir una vez. Esto se complementa con el endpoint dedicado de reimpresión para dejar registro explícito del motivo.

3. Reimpresión controlada (**reprintTicket**)


```

POST /api/tickets/:id/reprint { motivo }
  |
  ▼
Validar motivo obligatorio
  |
  ▼
Buscar ticket → 404 si no existe
  |
  ▼
{ticket.impreso === false? —Sí→ 400 "No se puede reimprimir un ticket que no
ha sido impreso"
  | No (ya fue impreso antes)
  ▼
Agregar entrada al arreglo ticket.reimpresiones:
  { fecha: now, motivo, usuario: req.user._id, puntoTrabajo: req.user.puntoTrabajo
  }
  |
  ▼
ticket.save()
  |
  ▼
200 { ticket con historial de reimpresiones actualizado }

```

Acceso restringido exclusivamente a `authorize('jefe')` en `routes/tickets.js`, consistente con la regla de negocio validada desde la Semana 1 (RF-07).

4. Validación de datos de retiro (regla común a impresión y canje)

Campo	Regla
<code>quienRetira</code>	Obligatorio; debe ser <code>Titular</code> , <code>Titular Compra</code> u <code>Otro</code>
<code>celular</code>	Obligatorio siempre
<code>quienOtro</code>	Obligatorio solo si <code>quienRetira = "Otro"</code>
<code>parentesco</code>	Obligatorio solo si <code>quienRetira = "Otro"</code> ; valores permitidos: Esposo/a, Hijo/a, Padre/Madre, Hermano/a, Amigo/a, Otro parentesco

Esta validación existe tanto en el frontend (formulario, para feedback inmediato) como en el backend (controlador y esquema Mongoose con `validate` condicional), de modo que no se puede omitir enviando la petición directamente a la API.

5. Historial embebido de reimpresiones

Se optó por modelar `reimpresiones` como un arreglo de subdocumentos dentro del propio `Ticket` (no como una colección aparte), porque:

- Su ciclo de vida depende completamente del ticket padre (no tiene sentido una reimpresión "huérfana").
- Se consulta casi siempre junto con el ticket (al ver el detalle de un boleto), no de forma independiente.

- El volumen esperado de reimpresiones por ticket es bajo (excepcional, no la operación principal).

6. Conclusiones del día

Quedan implementados los flujos de impresión y reimpresión controlada, reutilizando las mismas reglas de validación de datos de retiro definidas para el canje, y manteniendo el historial de reimpresiones trazable dentro de cada boleto.

Observaciones: Sin observaciones.

Pruebas de Funcionalidades de Canje

Actividad N°: 25 **Fecha:** 03/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Cerrar la semana verificando funcionalmente la gestión/consulta de tickets, las búsquedas y filtros, y el proceso de canje e impresión desarrollados, incluyendo la verificación de las dos correcciones aplicadas esta semana.

2. Alcance de la verificación

Igual que en la Semana 4, esta verificación se realizó mediante **revisión funcional del código implementado**, trazando cada caso contra la implementación real. La ejecución en vivo contra un ambiente con datos reales queda para la Semana 7 (Integración y pruebas).

3. Casos de prueba — Consulta y filtros

ID	Caso de prueba	Resultado esperado	Estado
PC-01	Staff consulta <code>/api/tickets</code>	Solo recibe boletos de su propio <code>puntoTrabajo</code> , sin importar qué envíe el cliente	✓ Conforme
PC-02	Jefe consulta <code>/api/tickets?puntoTrabajo=X</code>	Recibe boletos filtrados por el punto de trabajo indicado	✓ Conforme
PC-03	Búsqueda general por nombre parcial	Devuelve coincidencias insensibles a mayúsculas	✓ Conforme
PC-04	Búsqueda por Ticket ID exacto	Devuelve único resultado exacto	✓ Conforme
PC-05	Filtro combinado búsqueda + localidad	Ambos filtros actúan como intersección (<code>\$and</code>), no se amplían los resultados	✓ Conforme
PC-06	Búsqueda por cédula	Coincide contra <code>'Numero de Cedula:'</code> ; ya no existe la condición muerta <code>'Número de Cédula: '</code> (limpiada esta semana)	✓ Conforme
PC-07	Staff con <code>puntoTrabajo</code> mal configurado (sin PuntoVenta activo asociado)	Antes del fix: recibía datos hardcoded de un evento anterior sin aviso. Después del fix: <code>404</code> con mensaje explícito de configuración faltante	✓ Conforme — verificado tras la corrección
PC-08	Paginación con <code>limit</code> mayor a 100	El sistema fuerza un máximo de 100 elementos por página	✓ Conforme

4. Casos de prueba — Canje individual y masivo

ID	Caso de prueba	Resultado esperado	Estado
PCJ-01	Canjear boleto sin celular	400 "Quien retira y celular son campos obligatorios"	✓ Conforme
PCJ-02	Canjear con quienRetira: "Otro" sin parentesco	400 "Debe especificar el parentesco..."	✓ Conforme
PCJ-03	Canjear boleto ya canjeado	400 "Este ticket ya fue canjeado"	✓ Conforme
PCJ-04	Canje masivo con lista mixta (algunos ya canjeados)	Procesa solo los pendientes; informa cuántos ya estaban canjeados	✓ Conforme
PCJ-05	Canje masivo con todos los tickets ya canjeados	400 "Todos los tickets (n) ya fueron canjeados previamente"	✓ Conforme
PCJ-06	Falla el registro de auditoría durante un canje	El canje se guarda igual; el error de auditoría se registra aparte sin revertir la operación	✓ Conforme
PCJ-07	Canje exitoso emite evento Socket.IO	Se emite ticket-updated a las salas de punto de venta y staff correspondientes	✓ Conforme

5. Casos de prueba — Impresión y reimpresión

ID	Caso de prueba	Resultado esperado	Estado
PI-01	Staff imprime un boleto ya impreso	400 "Este ticket ya fue impreso"	✓ Conforme
PI-02	Jefe reimprime desde /print un boleto ya impreso	Permitido (excepción explícita para rol Jefe)	✓ Conforme
PI-03	Reimprimir vía /reprint sin motivo	400 "Motivo de reimpresión es obligatorio"	✓ Conforme
PI-04	Reimprimir un boleto que nunca fue impreso	400 "No se puede reimprimir un ticket que no ha sido impreso"	✓ Conforme
PI-05	Staff intenta usar /reprint	403 Forbidden (solo Jefe)	✓ Conforme
PI-06	Reimpresión exitosa	Se agrega una entrada al arreglo reimpresiones con fecha, motivo, usuario y punto de trabajo	✓ Conforme

6. Resumen de hallazgos de la semana y su resolución

Hallazgo	Detectado en	Resolución
Fallback hardcodeado de localidades de un evento anterior en <code>getTicketsForStaff</code>	Día 21	✅ Corregido — ahora responde error explícito si el punto de trabajo no está configurado
Condición de búsqueda muerta ('Número de Cédula: ') en 3 controladores	Día 22	✅ Corregido — eliminada en <code>ticketController.js</code> y <code>puntoVentaController.js</code>

7. Conclusiones del día

Las funcionalidades de consulta, búsqueda, canje e impresión desarrolladas durante la semana cumplen con las reglas de negocio y criterios de aceptación definidos desde la etapa de requerimientos. Ambos hallazgos detectados durante el propio desarrollo de esta semana fueron corregidos y verificados en el mismo ciclo, sin quedar como deuda técnica pendiente.

Observaciones: Dos correcciones aplicadas y verificadas durante la semana; pendiente ejecución en vivo en ambiente de pruebas (Semana 7).